

Funciones de Activación en Deep Learning

Análisis Matemático del Gradiente
Descendente y Arquitectura de Red



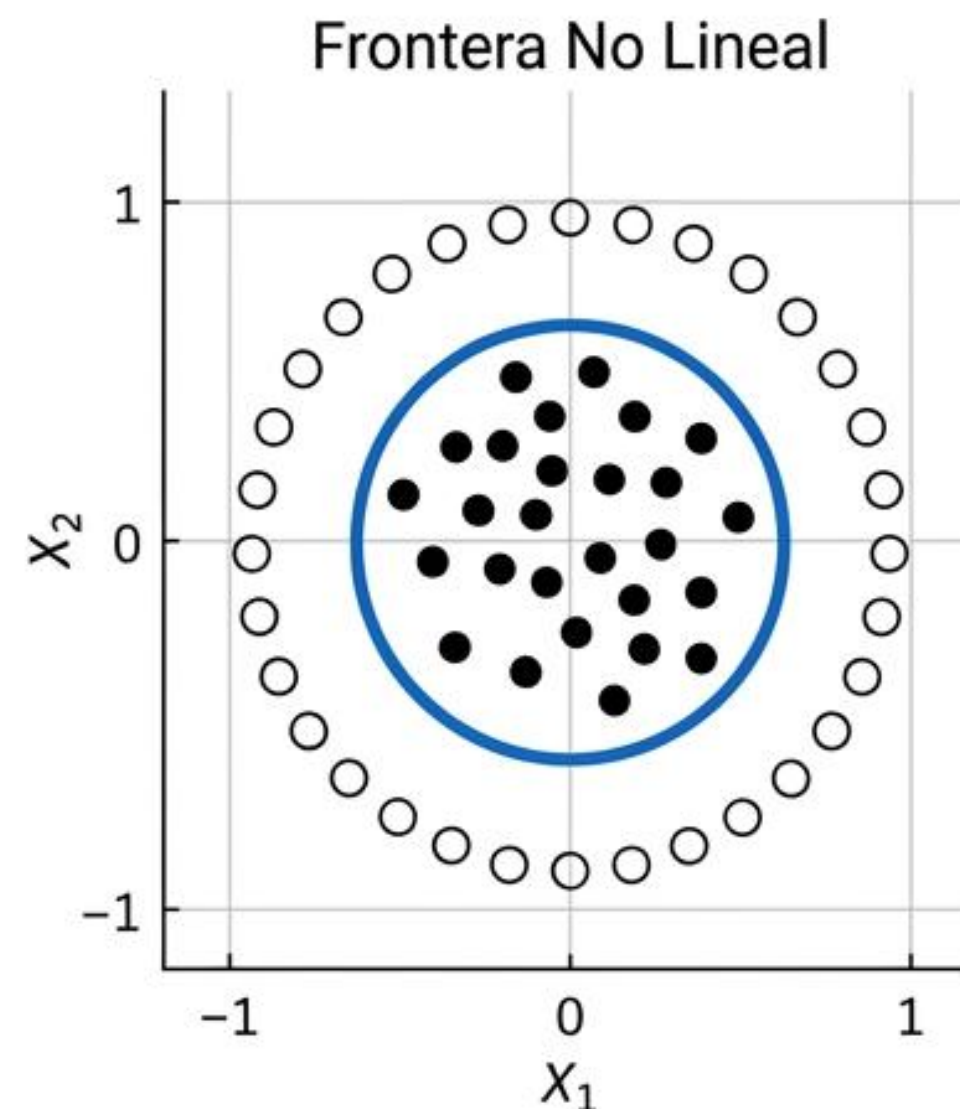
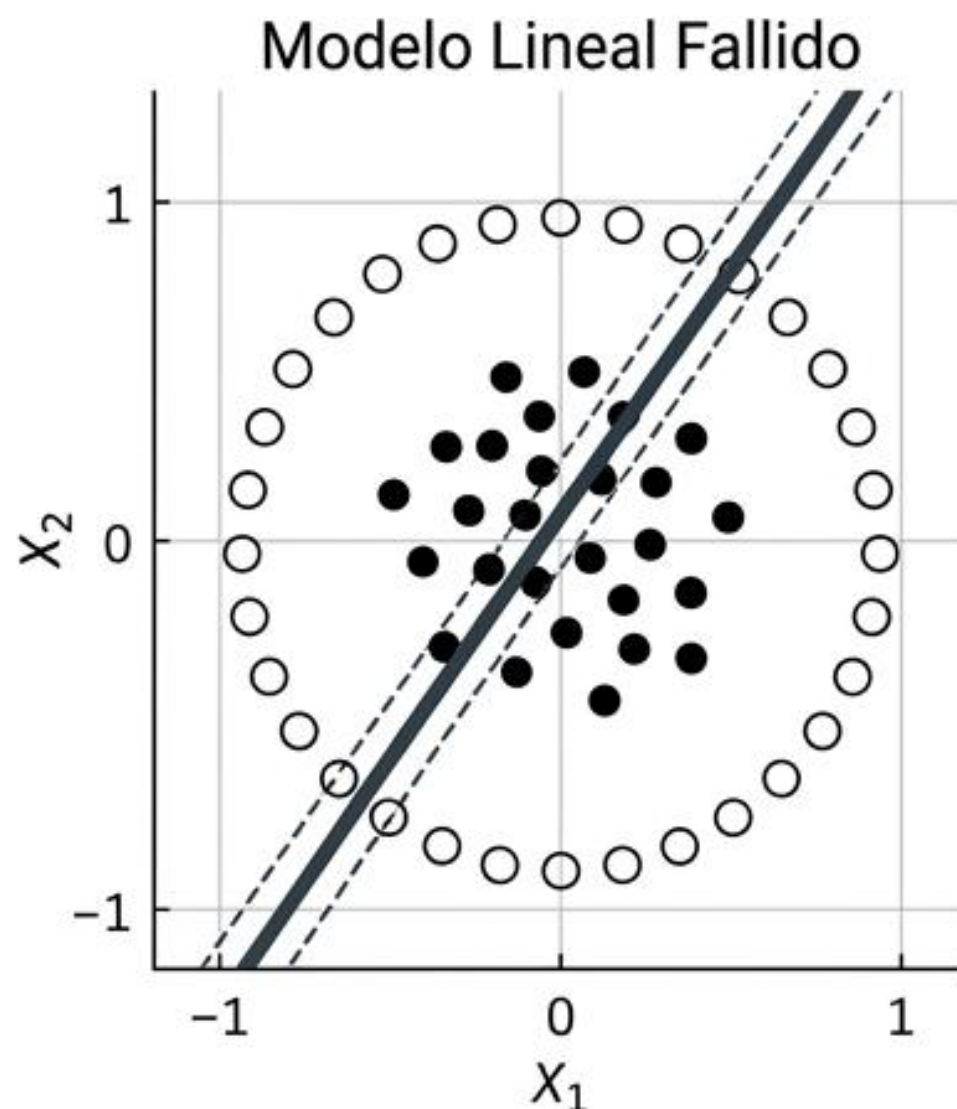
El Mandato de la No Linealidad

Aproximación Universal

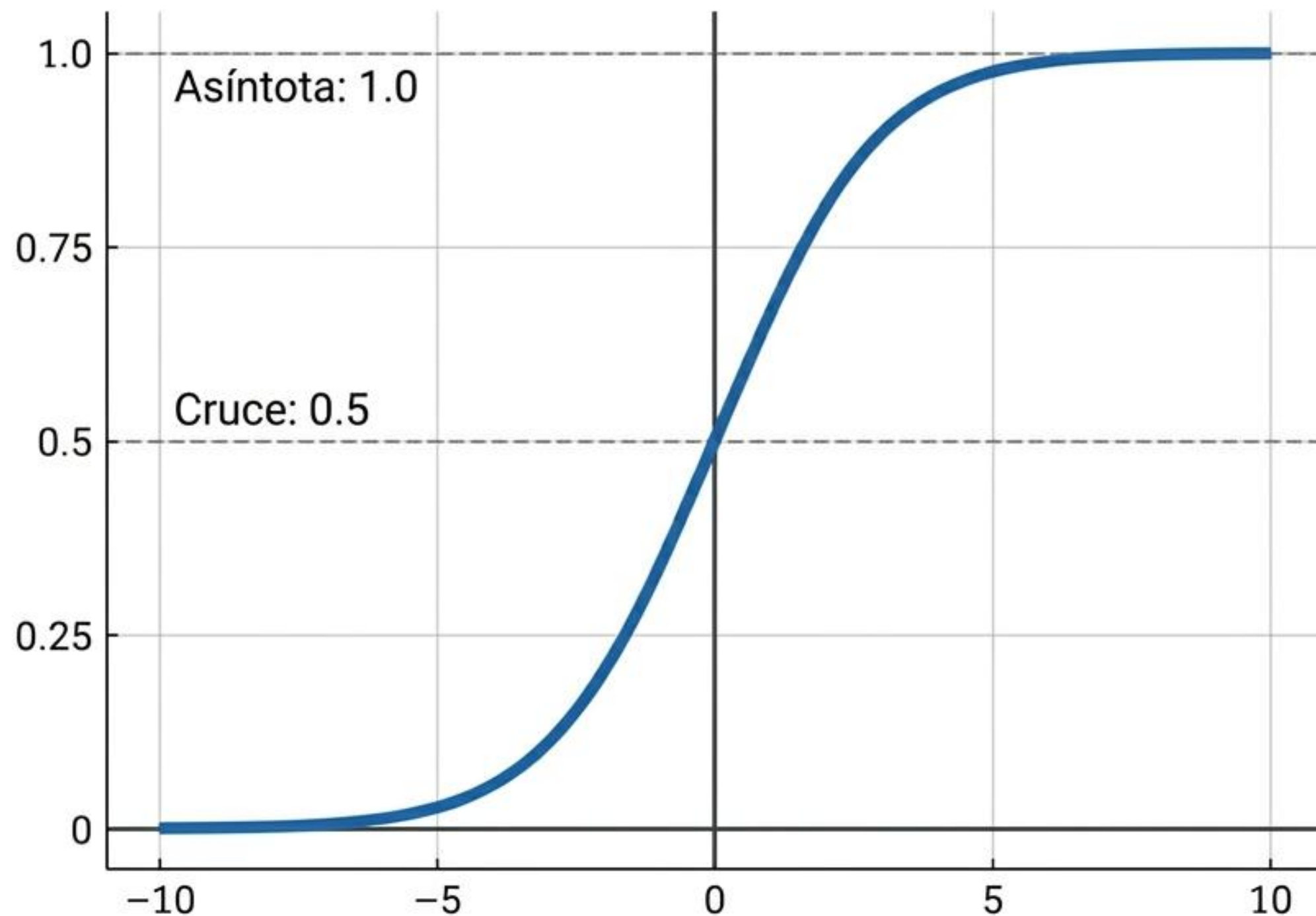
Para que una red neuronal actúe como un aproximador universal de funciones, es matemáticamente obligatorio el uso de funciones de activación no lineales.

El Colapso Lineal

Una red con activaciones puramente lineales colapsa en una única transformación afín, incapaz de resolver datos complejos.



La Función Sigmoideal



Definición Matemática:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Rango de Salida:

$[0, 1]$

Comportamiento:

Entradas Positivas -> Salida $[0.5, 1]$
Entradas Negativas -> Salida $[0, 0.5]$

Aplicación Principal:

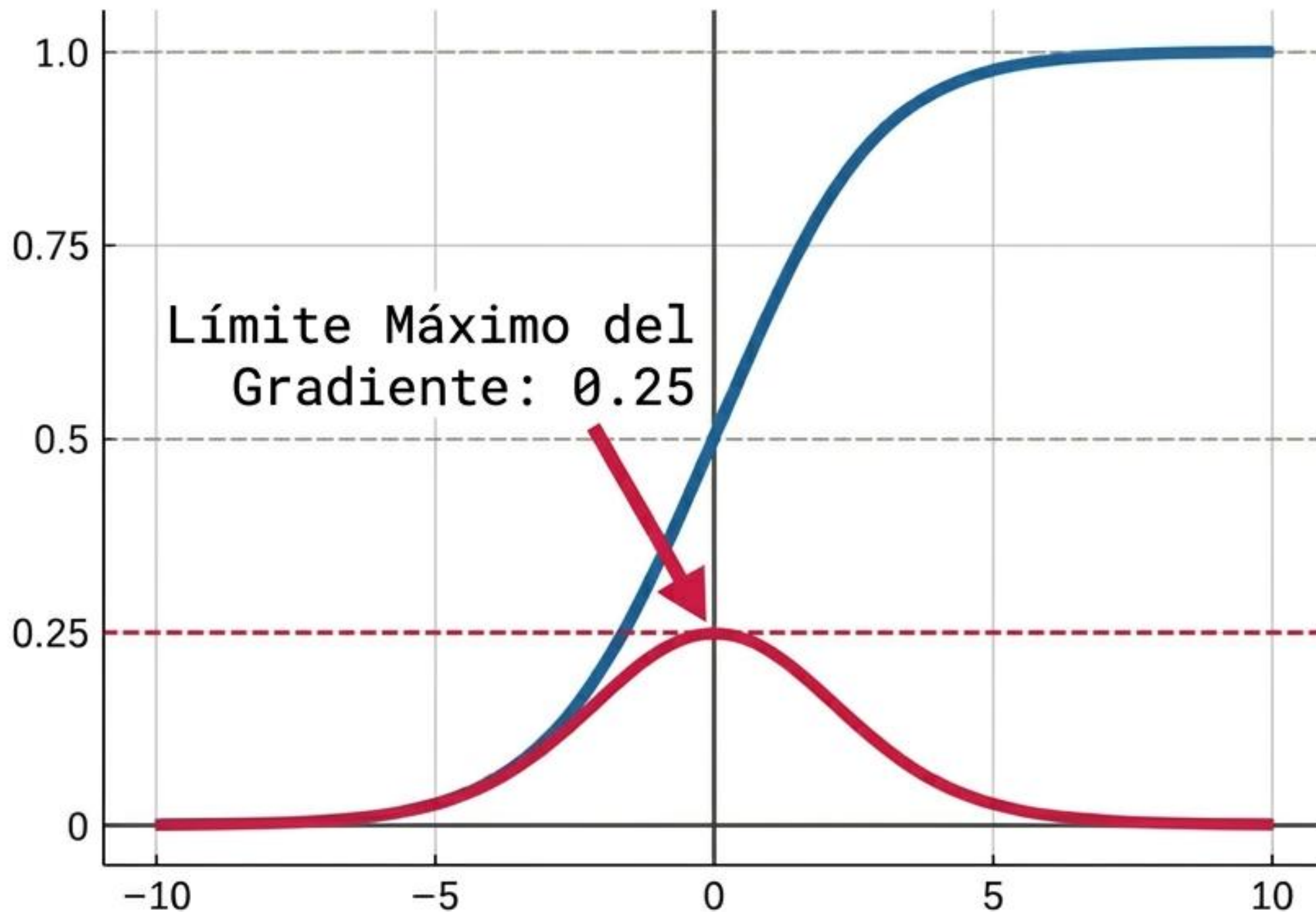
Probabilidades y clasificación binaria.

El Problema del Gradiente: Sigmoidal

El Límite del 0.25

En el mejor de los casos matemáticos, la derivada de la función sigmoidal jamás supera el valor de 0.25.

Para valores alejados de cero, la derivada colapsa a valores microscópicos.



Consecuencia: El Estancamiento del Aprendizaje

$$\Delta W = -\eta \cdot \frac{\partial L}{\partial y} \cdot \boxed{f'(x)} \cdot X$$

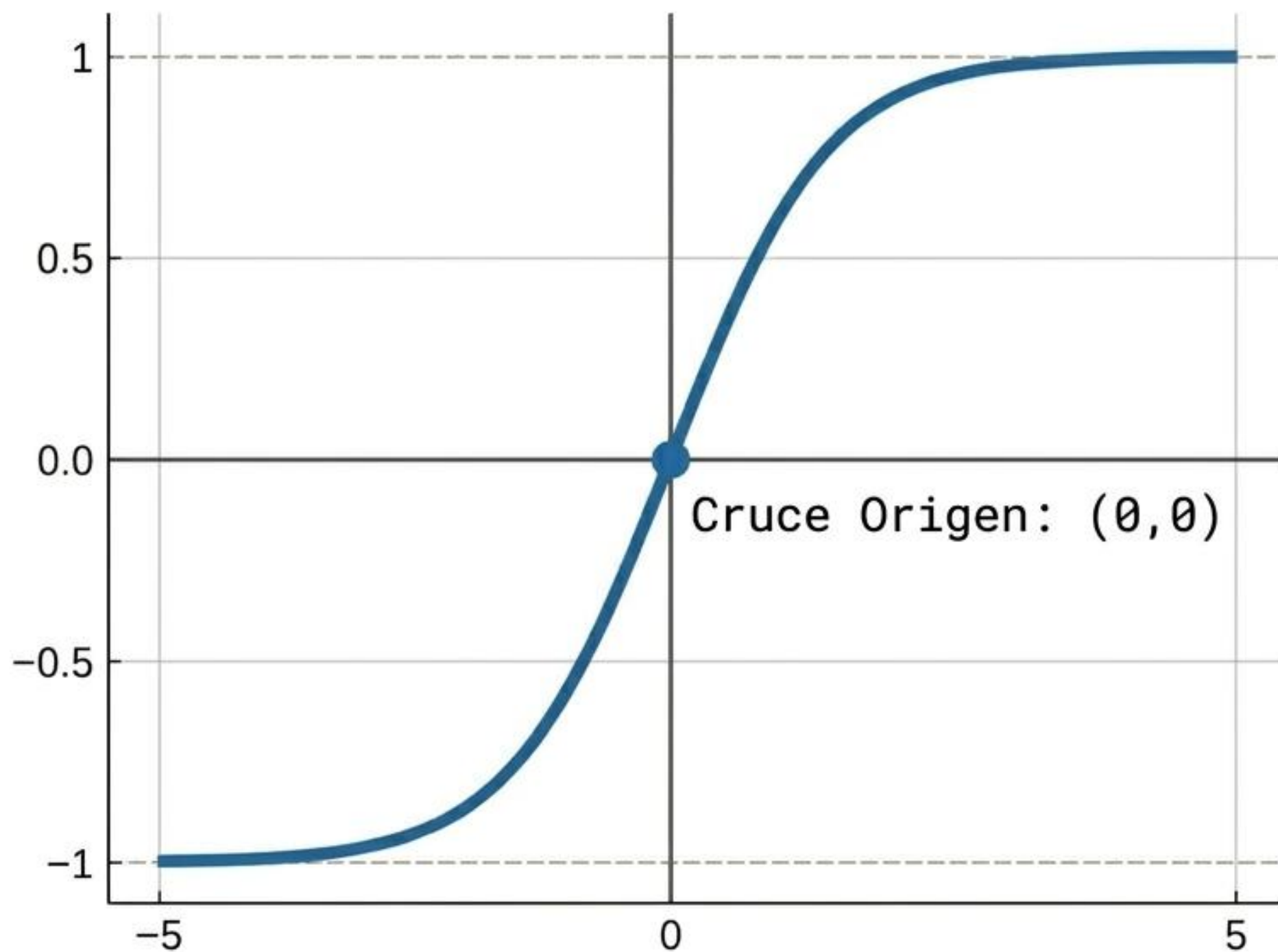
Multiplicador Máximo: 0.25

Señal de Error Original



Aprendizaje Lento: Dado que la ecuación depende multiplicativamente de esta derivada, cada paso de actualización **se reduce a, como máximo, una cuarta parte del error**. La red deja de aprender.

La Tangente Hiperbólica (Tanh)



Definición Matemática:

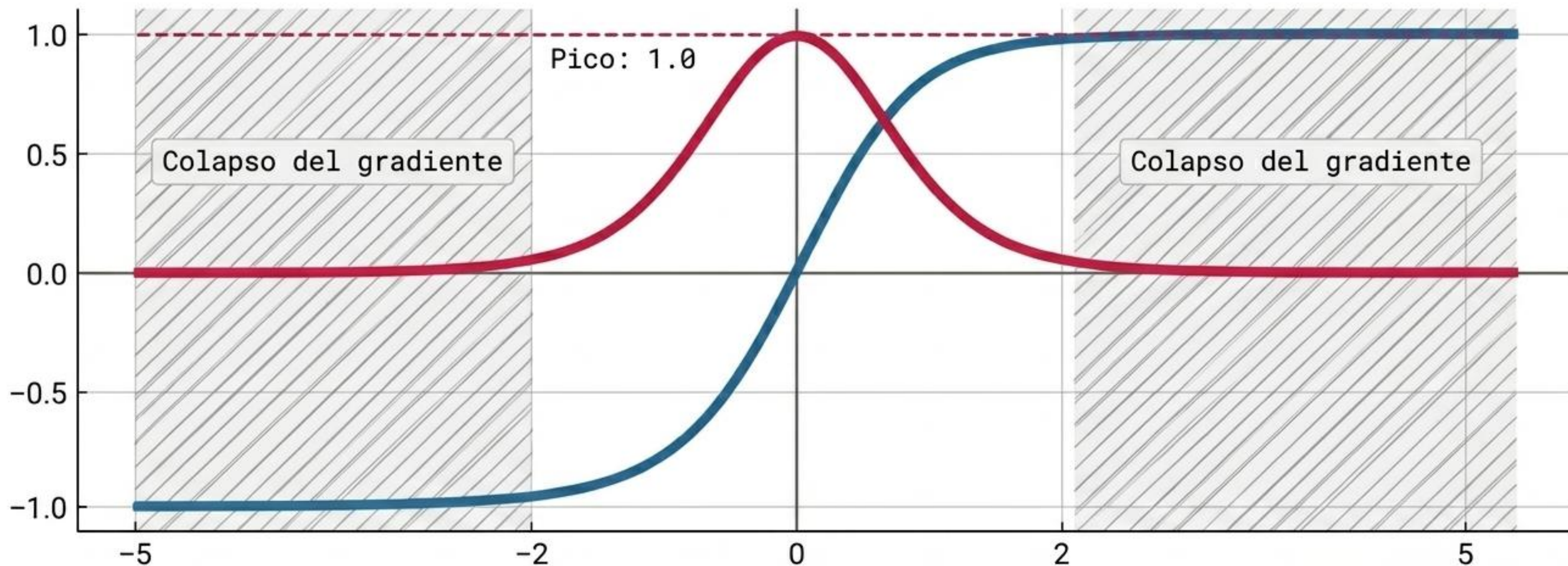
$$f(x) = \tanh(x)$$

Rango de Salida:
[-1, 1]

Características:

- Centrada en cero.
- Permite clasificación binaria polar.
- Ayuda a centrar los datos para la capa siguiente.

El Gradiente de Tanh: Un Problema Persistente

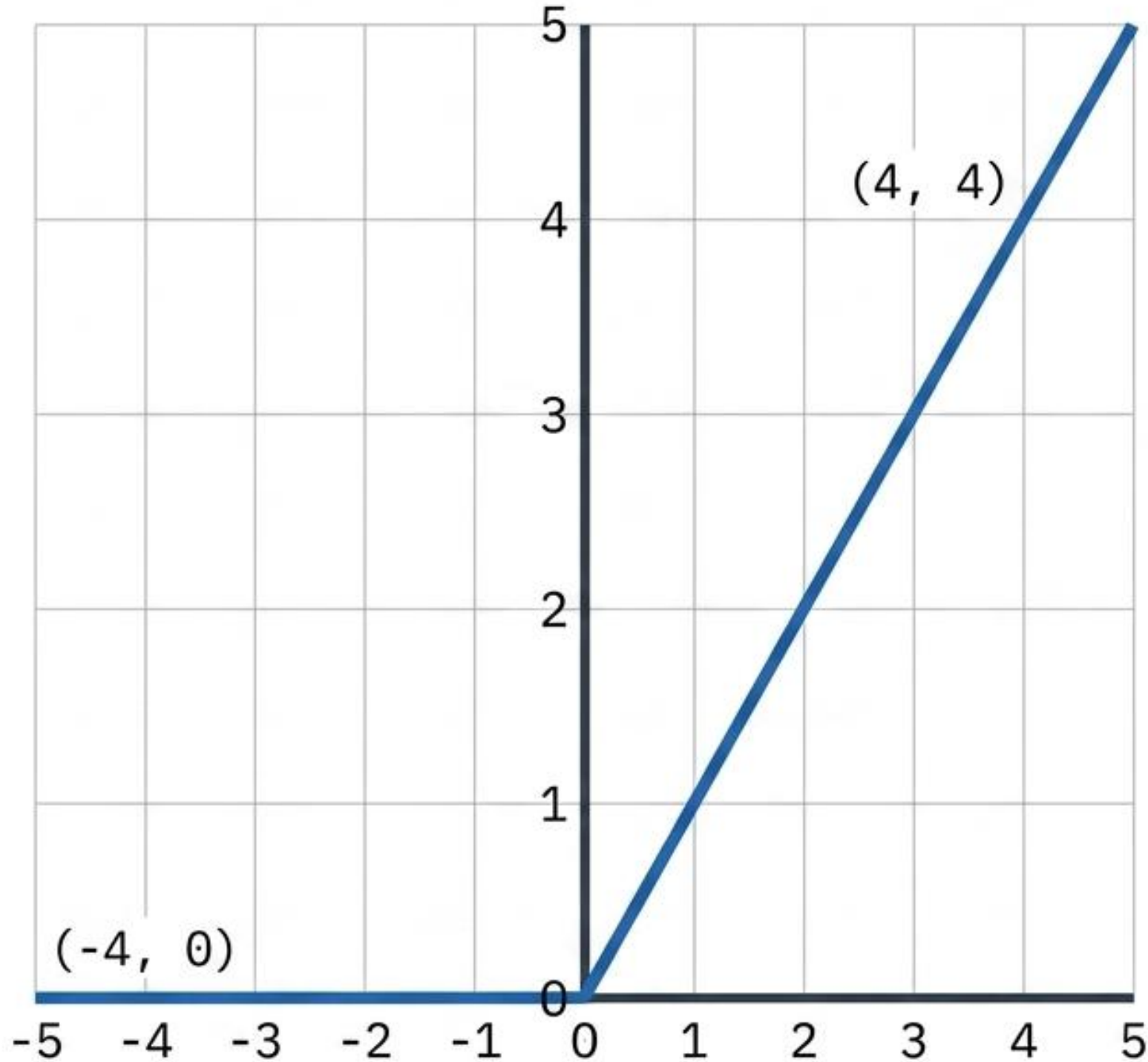


La Ilusión del Gradiente 1.0

Aunque la derivada alcanza 1.0, lo hace únicamente en un rango de valores extremadamente estrecho. Para el resto de las entradas, el gradiente decae rápidamente hacia cero.

Veredicto: El entrenamiento en redes profundas sigue siendo inaceptablemente lento.

La Solución Moderna: ReLU (Rectified Linear Unit)



Definición Matemática:

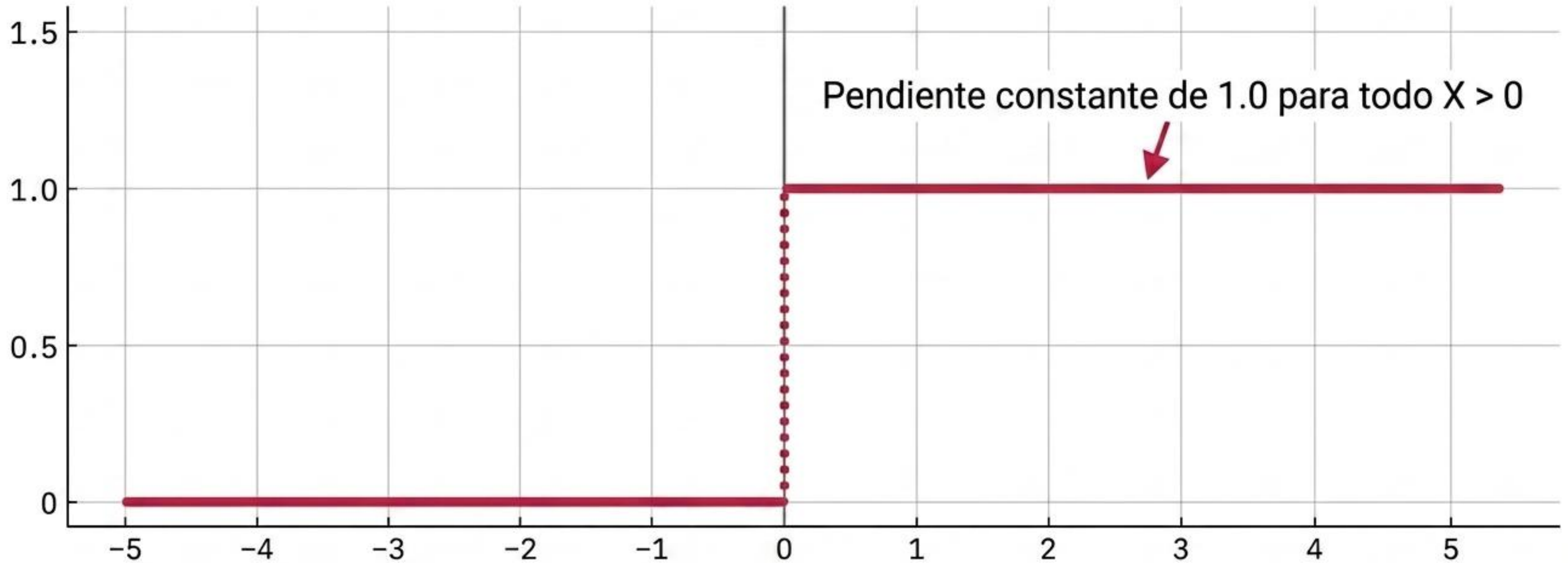
$$f(x) = \max(0, x)$$

Comportamiento por Secciones:

1. **Entradas Negativas:** Anuladas completamente (salida = 0).
2. **Entradas Positivas:** Pasan sin modificación (salida = X).

Nota: A pesar de estar compuesta por segmentos rectos, en su conjunto es una función no lineal, cumpliendo el requisito para aproximación universal.

La Derivada de ReLU: Flujo Desbloqueado



A diferencia de las curvas en forma de campana, la derivada de ReLU no decae.
Para cualquier entrada positiva, la derivada es exactamente 1.0.

Por Qué ReLU Acelera el Aprendizaje

El Cuello de Botella (Sigmoidal)

$$\Delta W = -\eta \cdot \left(\frac{\partial L}{\partial y} \right) \cdot \boxed{0.25} \cdot X$$

ΔW

Multiplicador Máximo: 0.25

Resultado: Disipación de la señal de error hacia las capas interiores.

La Autopista (ReLU)

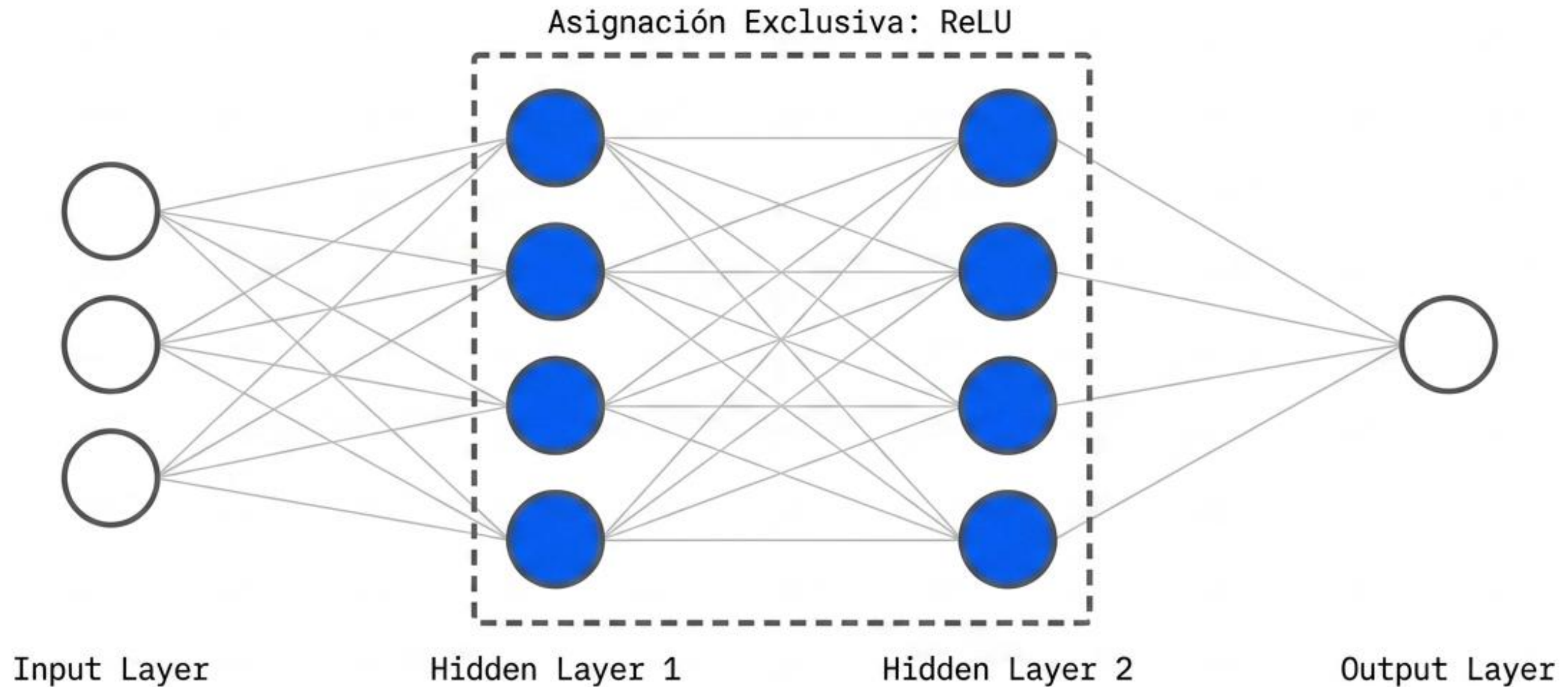
$$\Delta W = -\eta \cdot \left(\frac{\partial L}{\partial y} \right) \cdot \boxed{1.0} \cdot X$$

ΔW

Multiplicador Constante: 1.0 (para $X > 0$)

Resultado: El error retropropaga sin perder magnitud, acelerando drásticamente el ajuste de pesos.

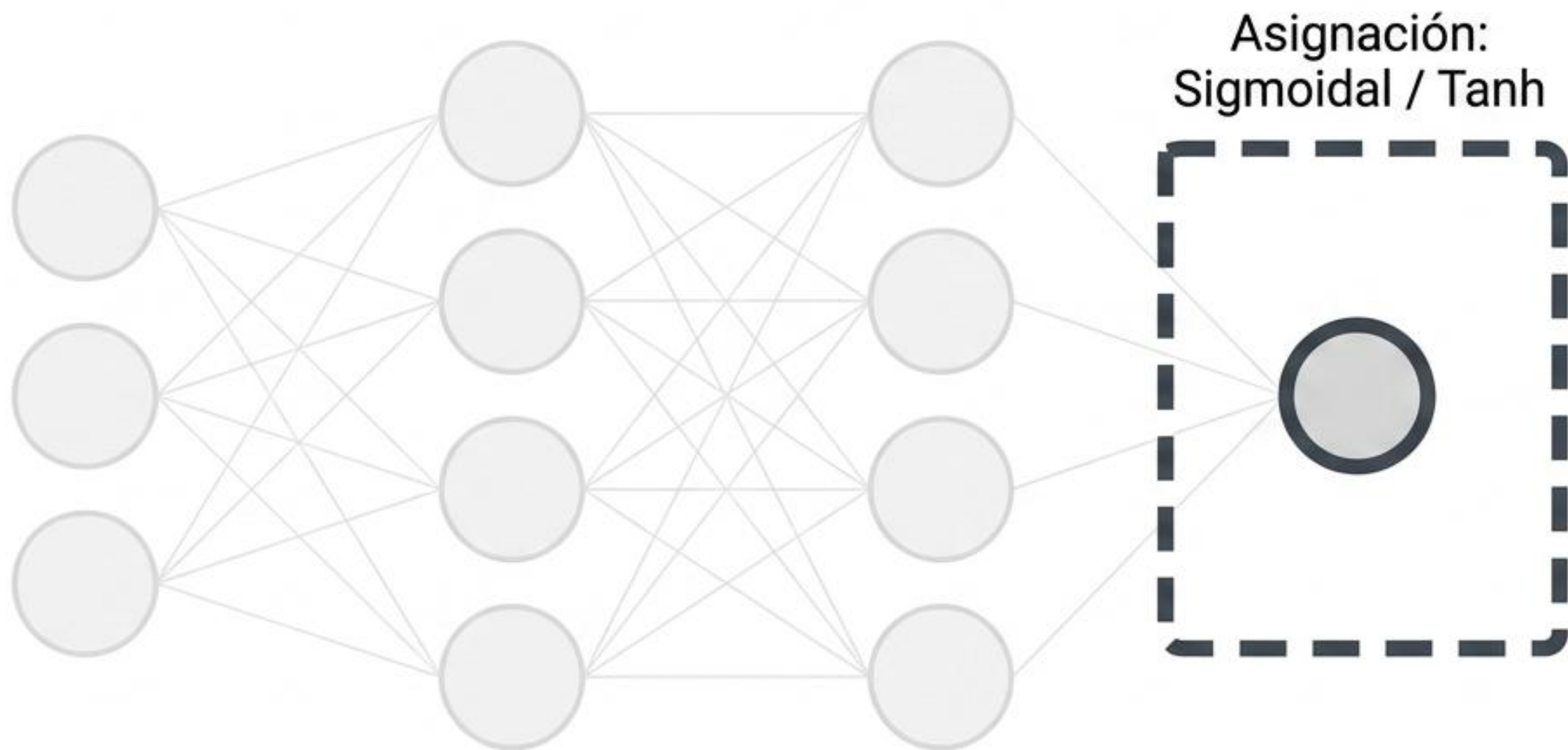
Arquitectura: Dominio de las Capas Ocultas



Por qué: En el interior de la red no existe la necesidad matemática de acotar o clasificar valores.

Ventaja: Dado que ReLU no tiene límite superior (rango $[0, \infty)$), su única función es propagar gradientes robustos (1.0) y procesar transformaciones no lineales a máxima velocidad.

Arquitectura: El Mandato de la Capa de Salida



Por qué: La predicción final exige un formato acotado e interpretable.

Ventaja: Comprime la salida a rangos estrictos ($[0, 1]$ para probabilidad, o $[-1, 1]$ polar).

Regla estricta: Jamás utilizar Sigmoidal o Tanh en las capas ocultas profundas.

Matriz de Decisión Arquitectónica

Función	Rango Matemático	Gradiente Máximo	Impacto en Velocidad	Capa Asignada
Sigmoidal	$[0, 1]$	0.25	Riesgo de Estancamiento	Capa de Salida (Clasificación binaria)
Tanh	$[-1, 1]$	1.0 (Rango muy estrecho)	Lento en redes profundas	Capa de Salida (Clasificación polar)
ReLU	$[0, \infty)$	1.0 (Constante si $X > 0$)	Aceleración Máxima	Capas Ocultas (Procesamiento interno)

El Siguiente Paso:

Con las piezas arquitectónicas (Sigmoidal, Tanh, ReLU)
correctamente ubicadas en la red...

¿Cómo ejecutamos el entrenamiento?

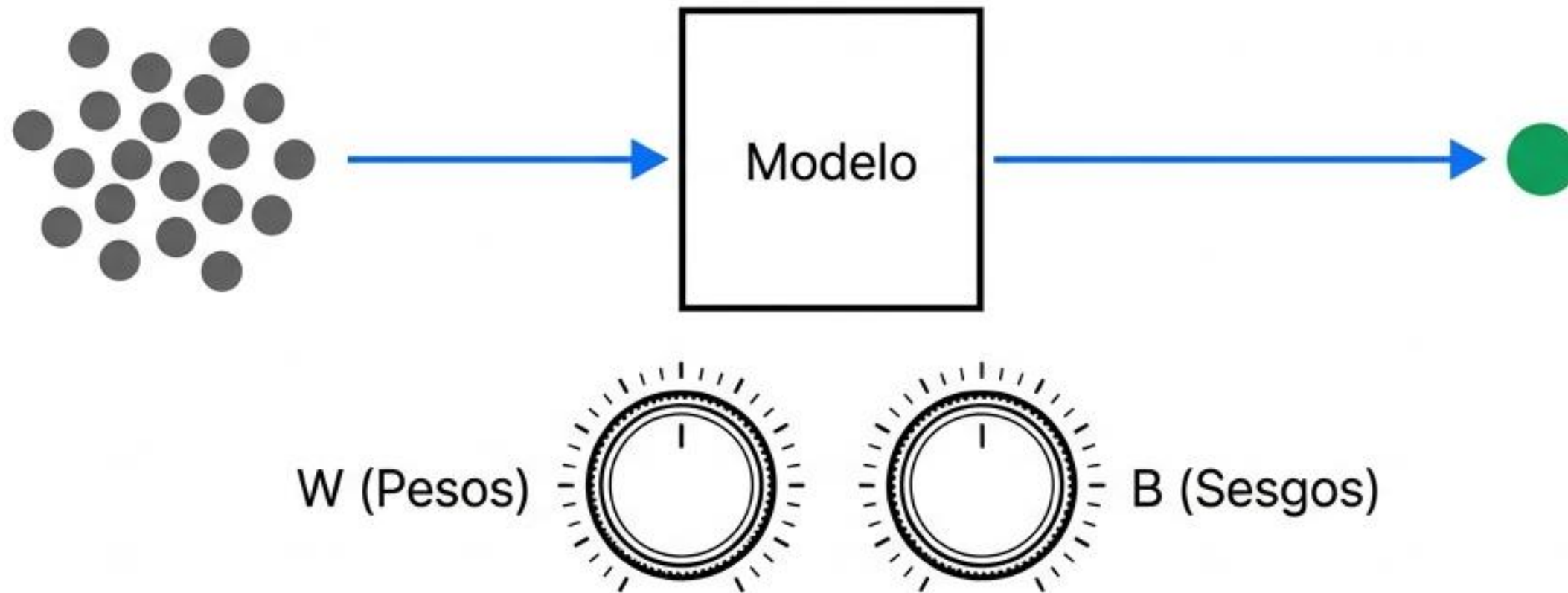
Próxima sesión: Algoritmos de optimización y entrenamiento en Deep Learning.

La mecánica interna del aprendizaje profundo.

Una radiografía visual de **Forward y Backward Propagation** en Redes Neuronales.

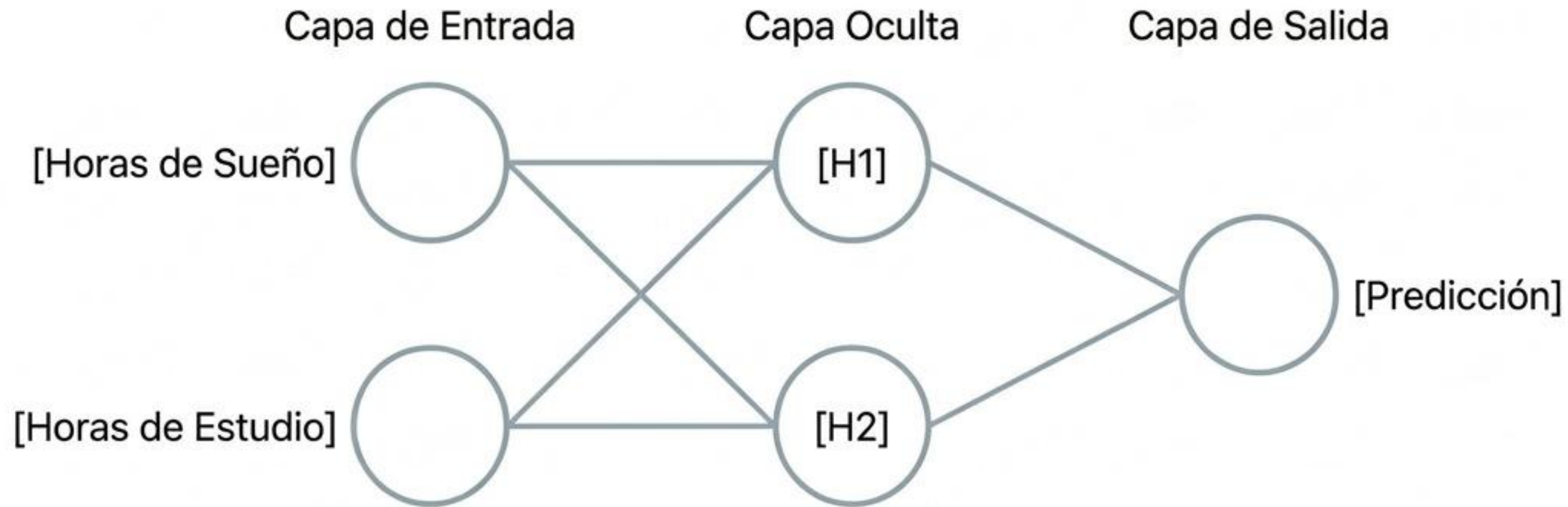
El objetivo fundamental es la minimización del error.

Entrenar una red neuronal es, en esencia, un proceso de ajuste automático. El algoritmo busca iterativamente los parámetros exactos —los coeficientes W y los sesgos B — que reduzcan al máximo la función de pérdida.



Al principio, la red ignora cómo resolver el problema. El entrenamiento es el proceso de afinar estos diales hasta encontrar la configuración perfecta.

La arquitectura de nuestro modelo base.



Capa de Entrada:

Recibe los datos crudos del entorno.

Capa Oculta:

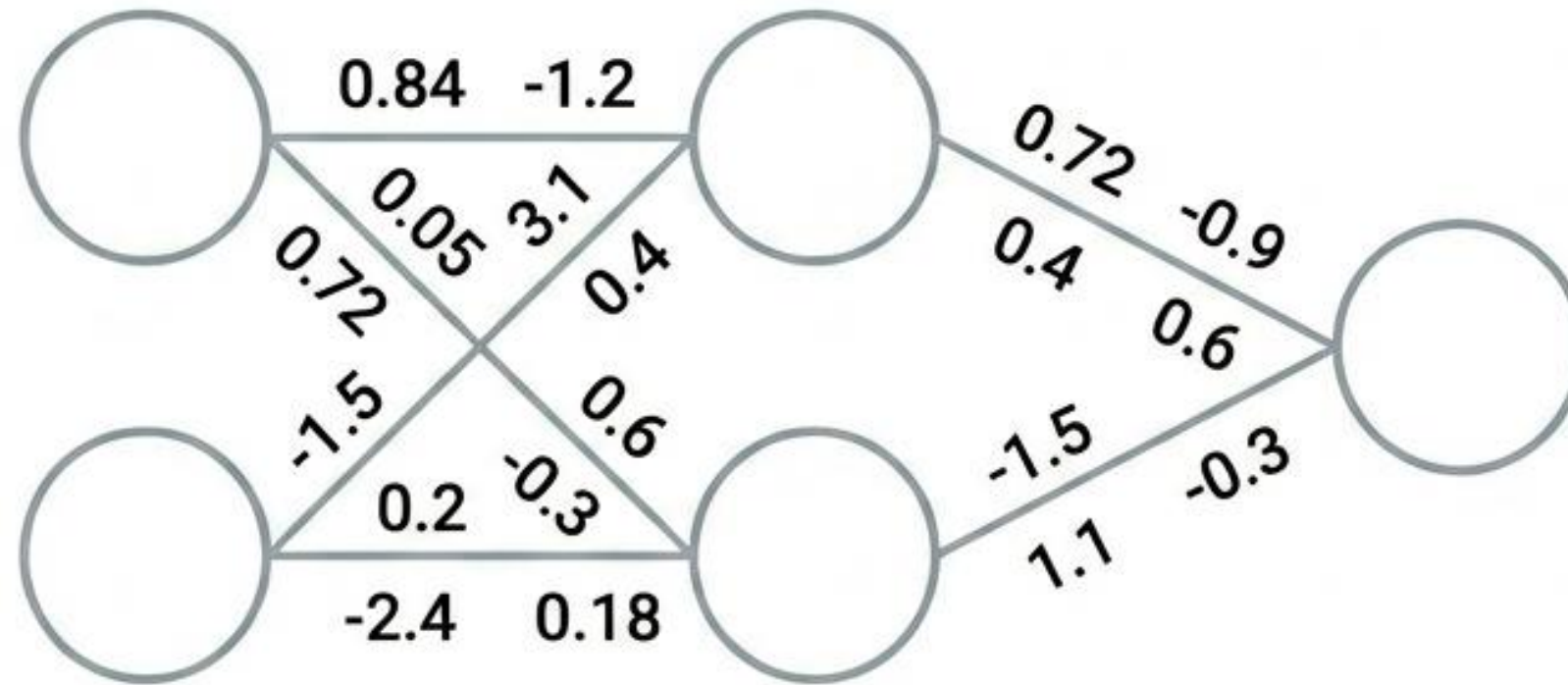
Realiza la extracción de características complejas mediante interacciones.

Capa de Salida:

Comprime la información para emitir una clasificación final.

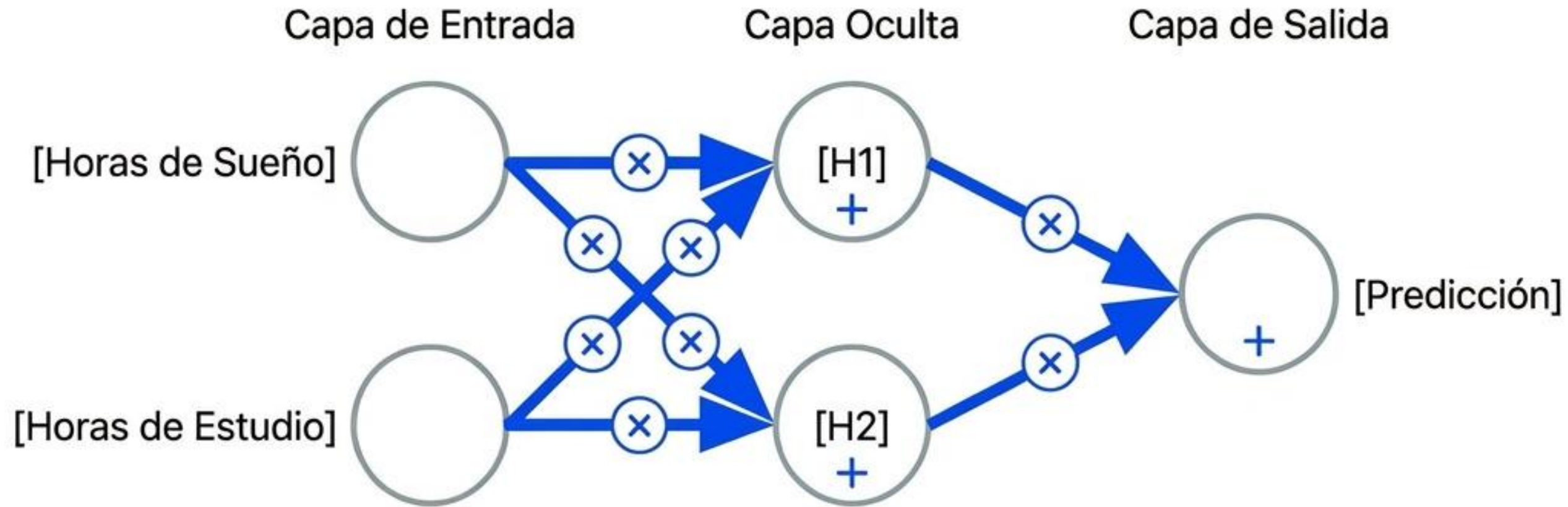
El punto de partida es el desconocimiento total.

Antes del primer ciclo de entrenamiento, los parámetros W y B nacen con valores completamente aleatorios. La red neuronal es una "pizarra en blanco", lo que garantiza que sus primeras predicciones serán erróneas.



La inicialización aleatoria es necesaria para romper la simetría y permitir que cada neurona aprenda patrones distintos.

Forward Propagation ejecuta el flujo de datos.



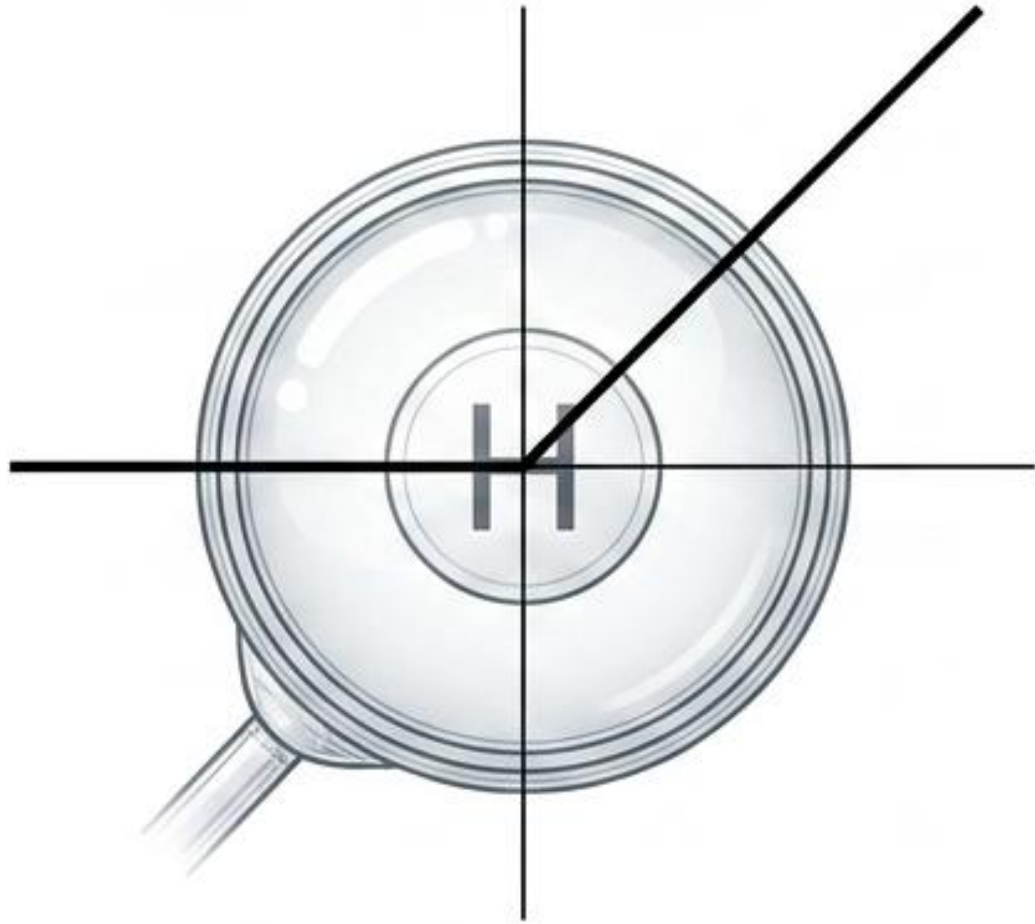
1. Entrada: Los datos ingresan a la red.

2. Transformación Lineal: Se multiplican las entradas por los pesos aleatorios (W) y se suma el sesgo (B).

3. Activación: La señal cruza hacia la siguiente capa.

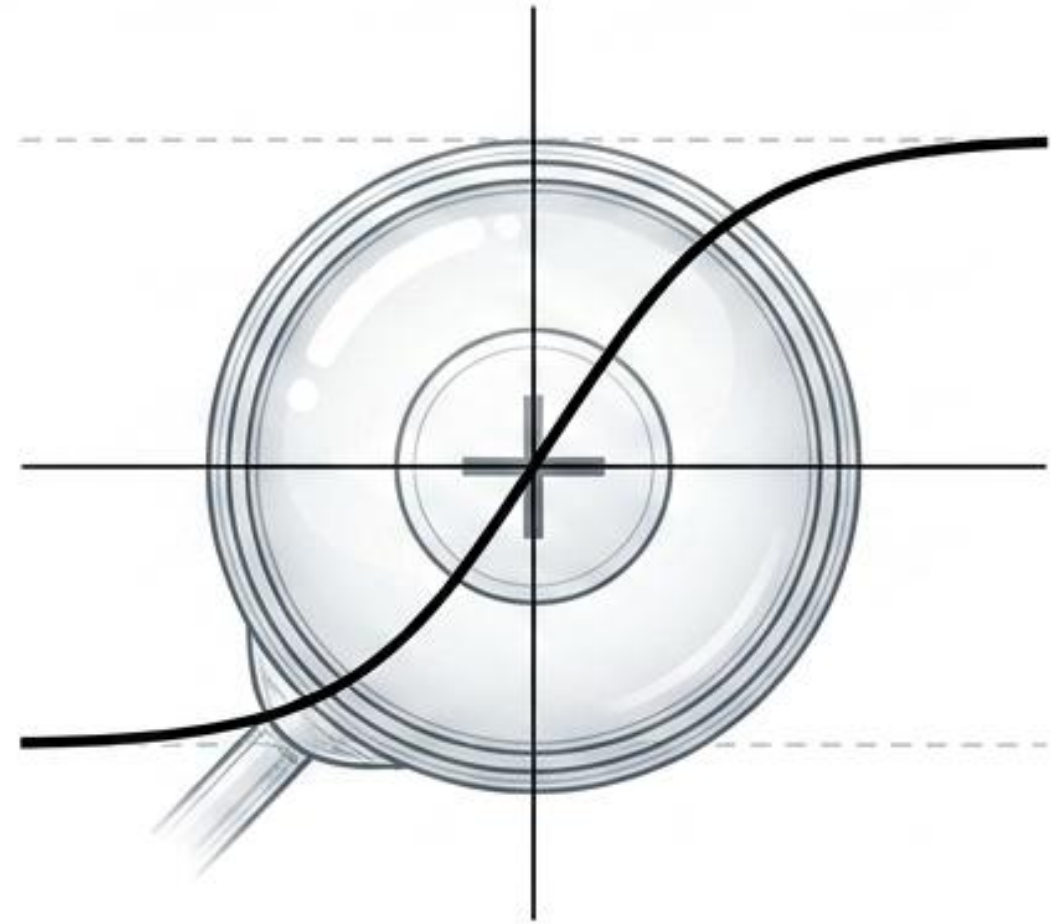
Nos movemos estrictamente de izquierda a derecha para mapear las entradas hasta generar una salida.

Las funciones de activación introducen la no-linealidad.



Capas Ocultas:

Usamos ReLU. Apaga las neuronas inactivas (valores negativos a cero) y permite a la red modelar patrones matemáticos altamente complejos.



Capa de Salida:

Usamos la función Sigmoidal. Comprime el resultado final matemático en un valor estricto entre 0 y 1, ideal para tareas de clasificación (ej. 0 = Reprobado, 1 = Aprobado).

El cálculo de la pérdida mide la magnitud de nuestro fracaso.

Forward Propagation termina aquí.
La predicción final de la red se compara directamente con la categoría real del dato.

Esta diferencia matemática es nuestra Función de Pérdida.

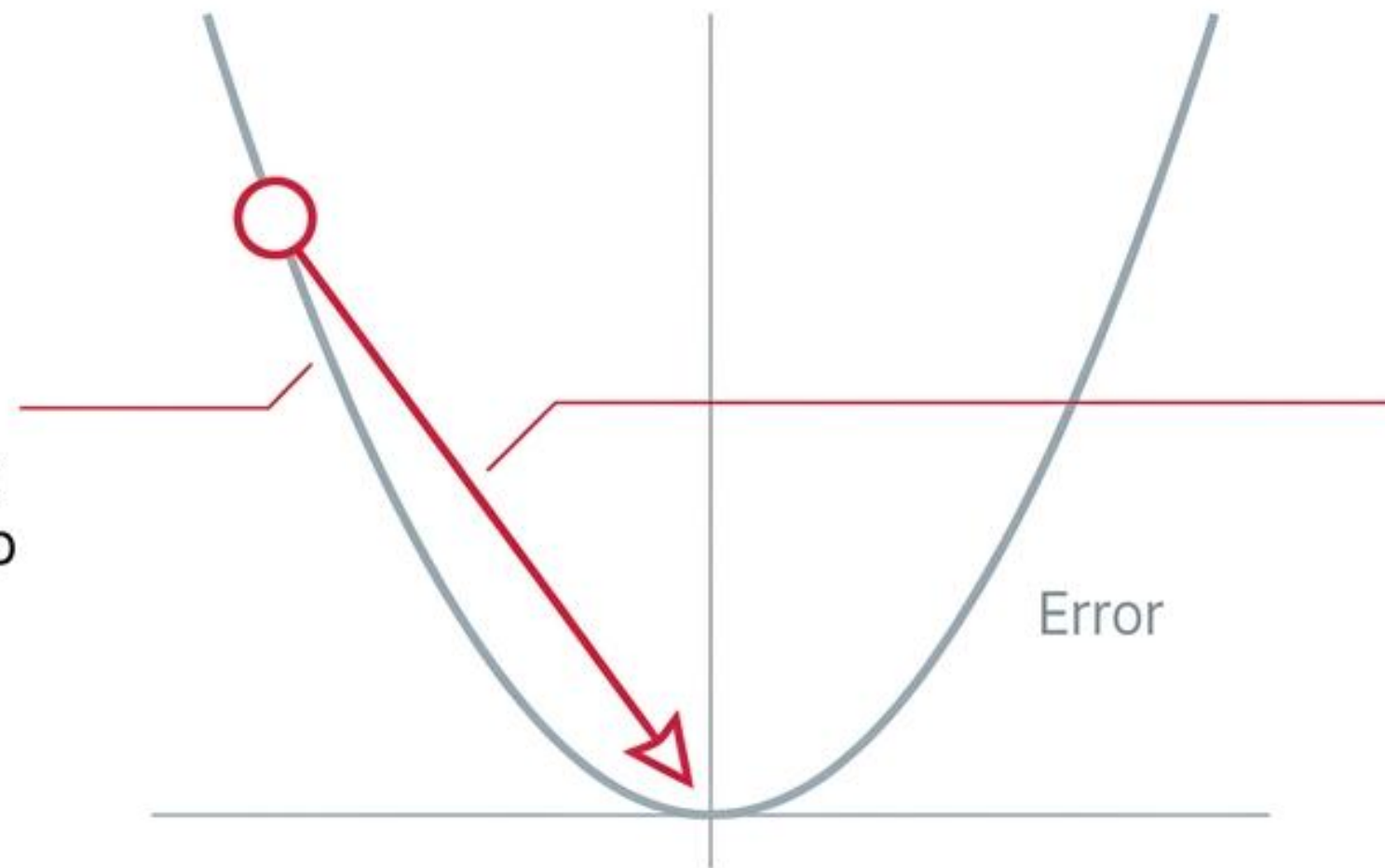


Toda la inteligencia artificial se basa en un principio: este error rojo debe reducirse a cero.

El Gradiente Descendente es el motor de corrección.

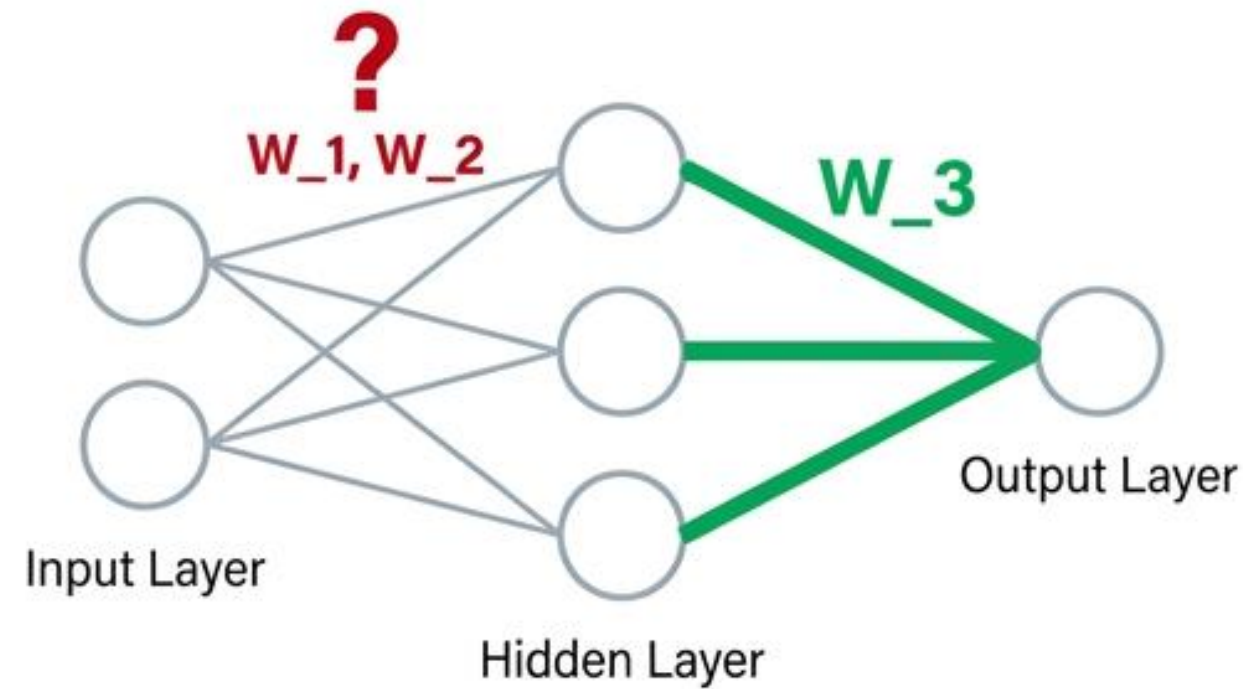
$$W_{nuevo} = W_{original} - \alpha \cdot \frac{\partial Error}{\partial W}$$

$\partial Error / \partial W$: La pendiente.
Nos indica en qué dirección
matemática está el aumento
del error. Nos movemos en
la dirección opuesta.



α (Tasa de Aprendizaje):
El tamaño del paso que
damos cuesta abajo.

El problema de la dependencia implícita.



El Camino Fácil

Actualizar los coeficientes de la capa de salida es directo. El Error depende explícitamente de esa última predicción.

El Misterio

¿Cómo actualizamos los coeficientes de la primera capa? El Error no está conectado directamente a ellos. A simple vista, la pérdida parece ignorar lo que ocurre en las capas ocultas.

La Regla de la Cadena resuelve el misterio conectando las capas.

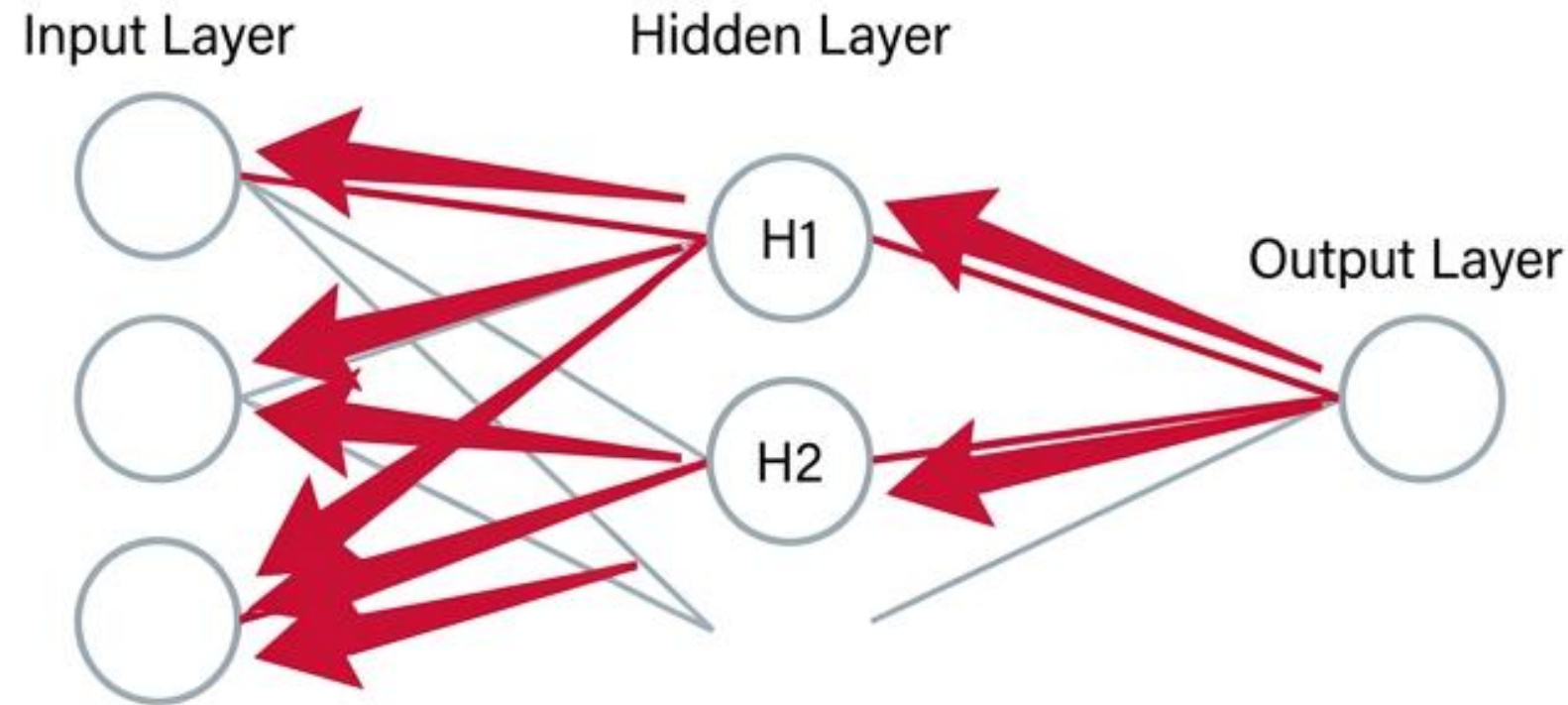


$$\frac{\partial Error}{\partial W_1} = \frac{\partial Error}{\partial Salida} \cdot \frac{\partial Salida}{\partial H_1} \cdot \frac{\partial H_1}{\partial W_1}$$

Descomponemos la derivada en pasos encadenados. El error depende de la salida, la salida depende de la capa oculta (H_1), y la capa oculta depende de sus pesos iniciales (W_1).

Matemáticamente, esto nos permite transferir la corrección a través de toda la red, sin importar cuán profunda sea.

Backward Propagation distribuye la corrección hacia atrás.

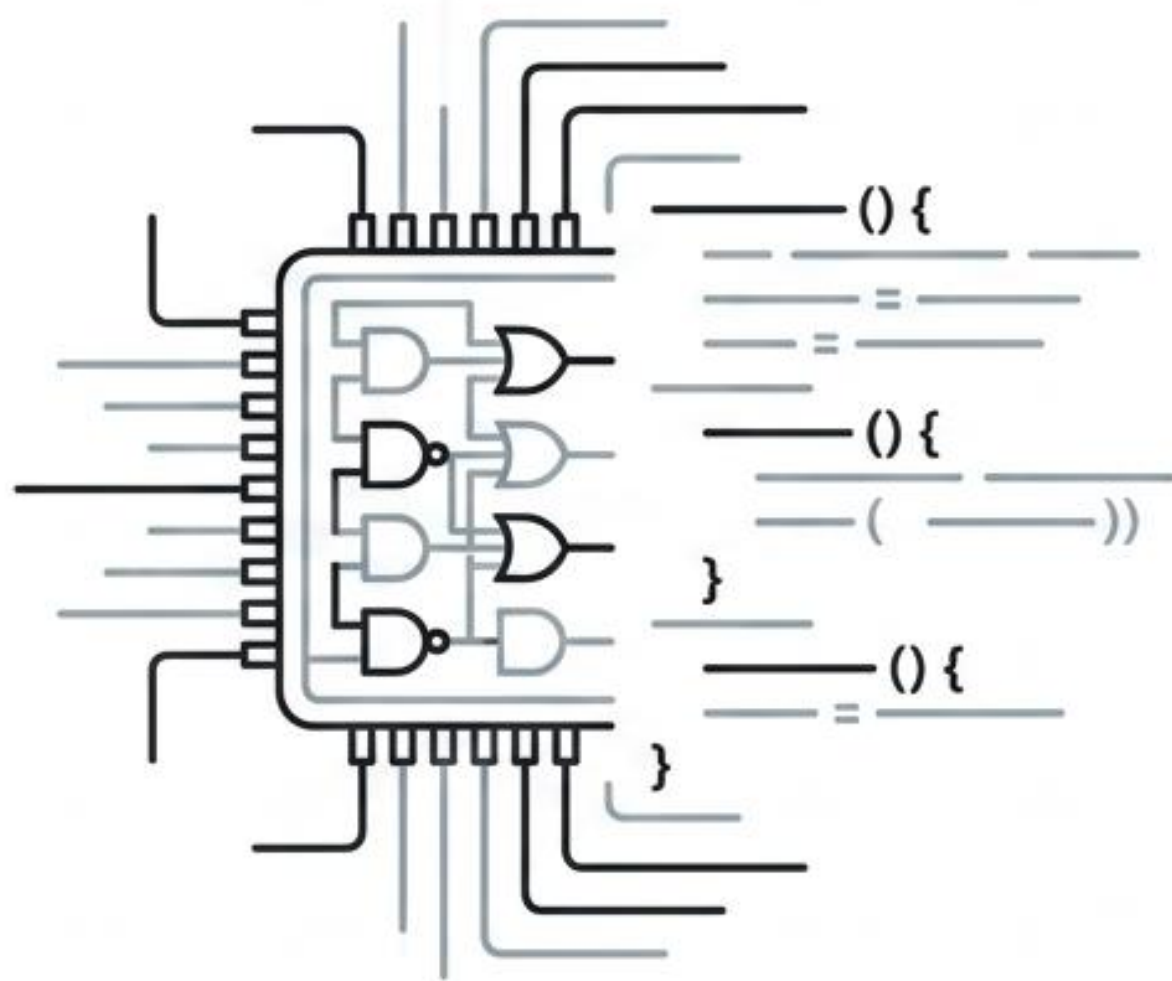


1. **Punto de Inflexión:** El error calculado en la salida inicia el retroceso.
2. **Propagación del Gradiente:** Usando la regla de la cadena, el algoritmo se mueve de derecha a izquierda.
3. **Actualización:** Se distribuye la culpa del error a cada neurona individual, actualizando finalmente todos los W y B.

La danza cíclica del entrenamiento neuronal.

	Forward Propagation (Flujo Azul)	Backward Propagation (Flujo Rojo)
Dirección	De Izquierda a Derecha	De Derecha a Izquierda
Propósito	Mapear entradas para generar predicciones	Propagar el error para corregir parámetros
Operación Principal	Sumas, multiplicaciones y Funciones de Activación	Cálculo de Derivadas y Regla de la Cadena
Estado Final	Genera la Función de Pérdida (Error)	Actualiza los pesos (W) y sesgos (B)

La automatización del cálculo en la práctica moderna.

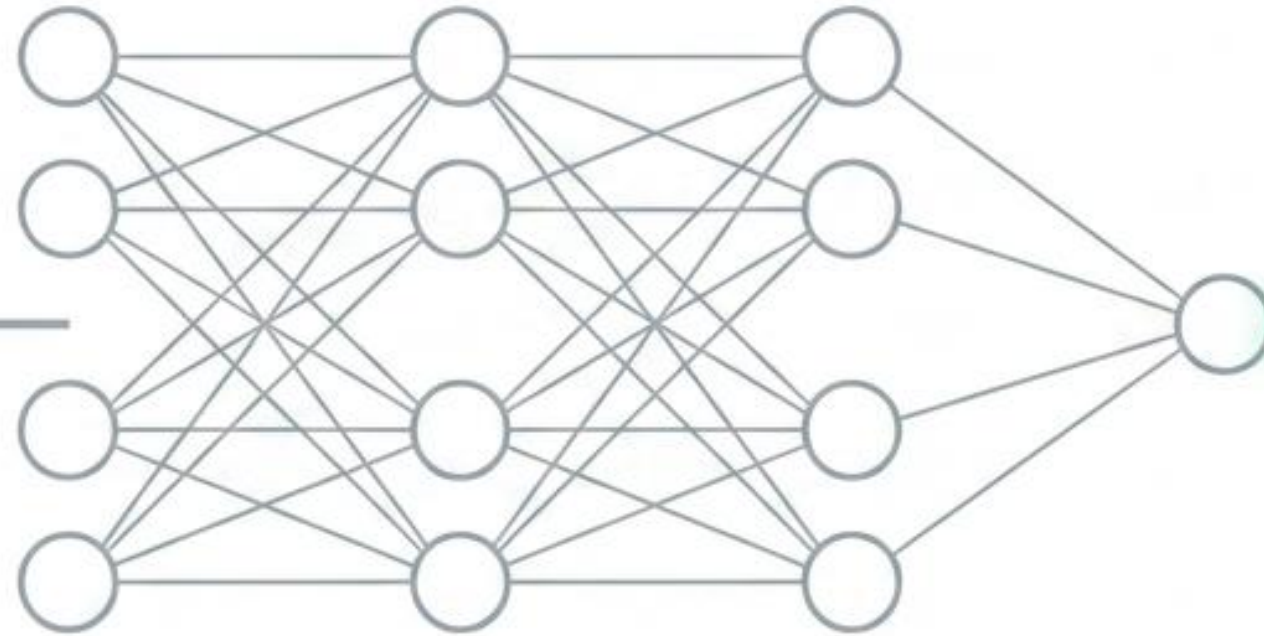


En la práctica diaria, no calculamos derivadas parciales a mano.

Frameworks modernos como **Keras** y **TensorFlow** implementan Backward y Forward Propagation internamente mediante diferenciación automática.

Sin embargo, entender esta matemática subyacente es lo que separa a un usuario básico de un arquitecto capaz de depurar y optimizar modelos complejos cuando fallan.

Del caos numérico al conocimiento estructurado.



Entrenar una red neuronal es un ejercicio de iteración disciplinada. A través de miles o millones de ciclos alternos de Forward y Backward Propagation, el modelo moldea sus parámetros ciegos hasta que su visión matemática refleja perfectamente la realidad.

El error se minimiza. La red ha aprendido.